



WORKSHOP · DEV ASSISTIDO POR IA

HARNESS ENGINEERING NA PRÁTICA

Por que o harness ao redor do modelo importa mais do que o modelo.

Felipe Rodrigues

Staff Engineer · BHub.ai

MAIO 2026



FELIPE RODRIGUES

- Staff Engineer @ BHub.ai
- AWS Community Builder
- Cursor Ambassador
- Mentor @ Tech Leads Club



LINKEDIN

linkedin.com/in/felipfr



AGENTES FALHAM. E VOCÊ NÃO PERCEBE.

Os quatro sintomas que separam demo de produção.

01

Vibe-codam

Recebem "implementa isso aí" e improvisam. Sem método, sem critério de pronto, sem nada que possa ser auditado.

02

Falham silenciosamente

Apagam um teste sem querer. "Completam" tarefas que continuam quebradas. Ninguém percebe até produção.

03

Queimam dinheiro

Loops invisíveis, retries cegos, modelo caro chamado para tarefa trivial. R\$ 200 em 10 minutos sem entregar nada.

04

Não dá pra explicar

Quando dá errado, você não tem como rebobinar a fita e mostrar o que aconteceu. Vira loteria, não engenharia.



AGENTE = MODELO + HARNESS

01

O MODELO

O cérebro. Lê, escreve, raciocina. É commodity – você troca de Claude pra GPT, pra Gemini e o mundo não acaba.

02

O HARNESS

O corpo, os reflexos, a sala de cirurgia. Decide quando o cérebro recebe contexto, quem confere o output.

03

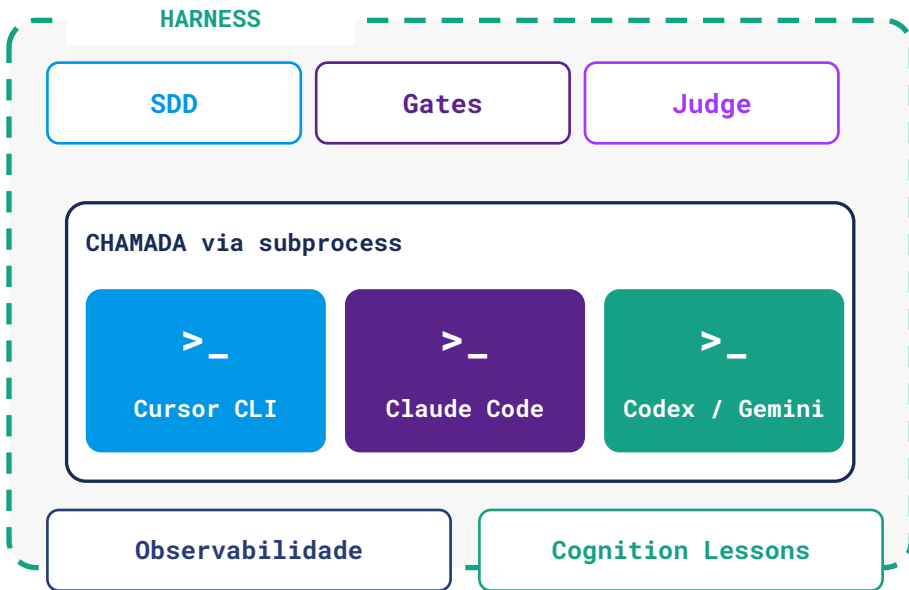
A CONSEQUÊNCIA

Sem harness, o cérebro não vira trabalho útil. Cirurgião brilhante em sala mal equipada não opera.



O HARNESS EM VOLTA DOS CLIS.

O modelo não é chamado direto. É invocado como subprocesso.



Por que subprocesso?

➤ Agnóstico de modelo

Hoje Cursor, amanhã Claude Code, depois Codex. Ou os tres juntos. O harness não muda.

➤ Isolamento de execução

Cada chamada vive em seu próprio processo – pode abortar, retomar, limitar.

➤ O harness vê tudo

Como ele invoca o CLI, o stdout/stderr passa por ele – gates, juiz e logs ficam por fora do modelo.

➤ Trocar é configuração

Mudar de modelo é mudar uma flag. Não há acoplamento à um vendor.



ONDE VOCÊ ESTÁ NESSE ESPECTRO?

Outside

the loop

Vibe coding

"Resolve aí" – e torce pro melhor.

RISCO

In

the loop

Micromanage

Revisa cada linha. Você vira o próprio gargalo.

GARGALO

On

the loop

Harness Engineering

Constrói e mantém o harness. Não conserta o código – muda o harness que produziu o código.

QUALIDADE

Flywheel

Self-improving

O harness aprende sozinho. Cada erro vira regra que entra na próxima rodada.

DIFERENCIAL

"A diferença entre 'in the loop' e 'on the loop' aparece quando o resultado não te agrada. Quem está sobre o loop muda o harness que produziu o artefato." – [KIEF MORRIS](#) · [THOUGHTWORKS](#)



TRÊS COISAS MUDARAM NO ÚLTIMO ANO.

01

MESMO MODELO, HARNESS DIFERENTE

No Terminal-Bench, dezenas de posições de movimento só pela troca do scaffold ao redor do mesmo LLM. Comparar dois agentes é comparar dois harnesses.

02

FRONTEIRA AINDA FALHA SEM DISCIPLINA

Anthropic, nov/2025: Opus 4.5 perde tarefas longas quando o harness não obriga feature_list, diário de progresso, commits incrementais. O modelo é forte; o que falta é harness.

03

OWASP ELEVOU A RISCO DE SEGURANÇA

Top 10 for Agentic Applications 2026: sandbox insuficiente, gates ausentes e fallback chains mal disparadas são risco de segurança, não mais só de qualidade.

TESE Quando você compara dois agentes, você não está comparando modelos. Está comparando harnesses.



PARTE 02 / 03

OS 7 PILARES

01 Spec-Driven Execution

02 Coordenação Pilot

03 Verification & Judge

04 Completion Gates

05 Guardrails

06 Resiliência

07 Observabilidade +
Cognition Lessons

bônus: sandbox



ESCREVE ANTES. EXECUTA DEPOIS.

O PROBLEMA

Agente recebe "implementa SSO". Sem spec, decide na hora se é SAML ou OIDC, cookie ou JWT, se cobre logout. Cada execução produz coisa diferente.

E o pior: o juiz que vai avaliar depois **não tem critério de aceitação** – porque o critério era a spec que nunca foi escrita.

O FLUXO



- A **spec** é o gabarito do design.
- O **design** é o gabarito das tarefas.
- As **tarefas**, com critérios de aceite, são o gabarito da execução.

COMO O SPEKTOR FAZ, NA PRÁTICA

O agente NÃO vai direto pro código. É forçado a passar pelas 4 fases, e cada fase produz um artefato que vira contrato da próxima. Tarefa trivial – bug fix de 3 linhas – a engine faz auto-sizing e pula fases automaticamente. SDD não vira burocracia.



UM MAESTRO, VÁRIOS INSTRUMENTOS.

O PROBLEMA

Agente recebe "migra autenticação pra OIDC, atualiza dashboard, faz documentação". Sem coordenação, ele empilha tudo numa única chamada de modelo, perde contexto no meio, esquece metade, comemora vitória entregando a parte fácil e some.

O PADRÃO



- **Planner** decide qual tarefa vem agora.
- **Executor** roda a tarefa.
- **Verifier** confere se ficou pronta.

COMO O SPEKTOR FAZ, NA PRÁTICA

Dois níveis hierárquicos. O PILOT é o maestro: decide a próxima intenção – rodar fase do SDD, chamar revisão, esperar input humano, abortar. O ORQUESTRADOR é a régua: pega cada tarefa, prepara contexto, chama o modelo, passa pelas gates, reporta de volta.



QUEM CONFERE O TRABALHO DO AGENTE?

O PROBLEMA

Agente recebe "adicionar logout no SSO". Faz refatoração que **deleta o login antigo, não adiciona logout, e os testes continuam passando** porque ninguém testava o caminho deletado. Sistema declara "sucesso" e segue.

"Testes verdes" não é prova de "tarefa pronta". É prova de "nada do que estava testado quebrou".

O CONCEITO

LLM-as-Judge. Um segundo modelo, separado do executor, olha o resultado **com base em critérios de aceitação** e decide se passa.

- **No planejamento:** o plano cobre os requisitos?
- **Na execução:** o entregue corresponde ao pedido?

Anti-padrão: mesmo modelo executando e julgando. Ele se autoaprova.

COMO O SPEKTOR FAZ, NA PRÁTICA

Judge calibrável: você regula o limiar de confiança, o número máximo de retentativas, e a política em caso de falha – RETRY, SKIP ou HALT. O juiz não é caixa-preta; é uma engrenagem que você afina por projeto.



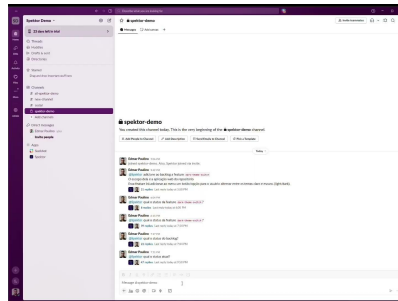
BARATO PRIMEIRO, CARO SÓ QUANDO PRECISA

O CONCEITO

Faithfulness = grau em que o entregue reflete a evidência: spec, código, ferramentas que ele alegou ter chamado. Agente pode entregar texto plausível e ser *infidel*: ignorou metade da spec, inventou chamada de ferramenta.

Embedding = "DNA matemático" do texto. Similaridade entre spec e output é rápida e barata – perfeita pra triagem antes do Judge LLM.

VÍDEO 01 – JUDGE EM AÇÃO



COMO O SPEKTOR FAZ, NA PRÁTICA

Embedding é o filtro grosso, quase de graça. Reprova direto o que claramente fugiu do escopo – saída cuja similaridade com a spec está lá embaixo. Não precisa chamar o Judge pra dizer o óbvio. Só casos duvidosos escalam pro juiz caro.

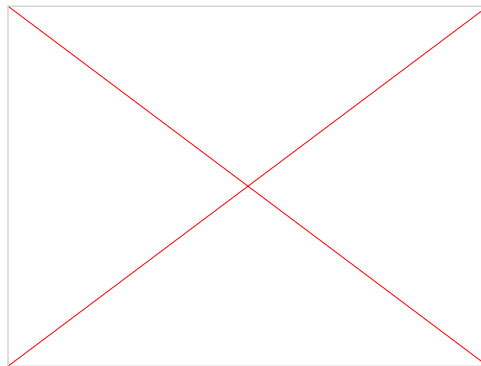


"PRONTO" NÃO É O AGENTE DIZER. É O SISTEMA PROVAR.

A CASCATA

Validação estrutural	<i>lint · testes · typecheck</i>
File Integrity	<i>não destruiu crítico?</i>
Change Sufficiency	<i>mudou o que foi pedido?</i>
Embedding Faithfulness	<i>"parece" a spec?</i>
Judge LLM	<i>cumpra a spec?</i>
TAREFA ACEITA	

VÍDEO 02



Sem isso: agente celebra vitória cedo. Recebe tarefa 8 num estado quebrado. Cinco tarefas depois, todo o trabalho é inútil.

COMO O SPEKTOR FAZ, NA PRÁTICA

Cascata explícita e ordenada. Cada etapa é MAIS CARA e MAIS PROFUNDA. O que cai no lint não chega no Judge – economiza dinheiro e tempo. Quando uma etapa falha, o motivo vem CLASSIFICADO e com SUGESTÃO ACIONÁVEL, não só "falhou".



OS LIMITES QUE O AGENTE NÃO PODE CRUZAR.

Não são sugestões. São portas trancadas.

FILE INTEGRITY

Arquivos que o agente **não pode reescrever livremente**: spec, testes que validam a tarefa, configs sensíveis. Sem isso, o agente resolve um teste que falha **apagando o teste**. Clássico.

CHANGE SUFFICIENCY

A mudança tem que ser **proporcional ao escopo**. Tarefa grande com diff de duas linhas é sinal vermelho. Pintor mostra dois cantos e cobra a casa inteira.

COST CAP

Cada execução tem orçamento explícito. Esgotou: para. Sem isso, agente entra em loop e queima **R\$ 200 em dez minutos**.

COMO O SPEKTOR FAZ, NA PRÁTICA

Cada guardrail vem por DEFAULT, com regras sensatas. Você não precisa lembrar de habilitar. Afina por projeto, por execução, por tarefa. Detalhe de campo: arquivos em JSON sofrem menos sobrescrita acidental que arquivos em Markdown (Anthropic, 2025).



MODELOS QUEBRAM. PROVEDORES QUEBRAM. TAREFAS TRAVAM.

OS 4 MECANISMOS

- **Stall** – agente parou de progredir. Mesma ferramenta em loop, output igual, stream parado.
- **Fallback chain** – cadeia entre modelos. Leve → forte → humano.
- **Escalation policy** – cadeia esgota: parar tudo ou pular-e-seguir.
- **Failure classification** – agente ou infra? Tratamento diferente.

CASO REAL · 2026

Um agente popular tinha fallback chain que **só disparava em erro de rate-limit**. Quando o stream travava (sem rate-limit), o agente ficava **15 minutos parado** sem nem tentar o fallback – gatilho errado.

Centenas de reais queimados por execução. Em silêncio.

A lição: o gatilho da fallback chain importa tanto quanto a chain. Stall e timeout precisam disparar fallback, não só rate-limit.

COMO O SPEKTOR FAZ, NA PRÁTICA

Resiliência é cinto, não corda só. DETECÇÃO DE ESTAGNAÇÃO observa progresso real, não só erros. FALLBACK CHAIN com múltiplos gatilhos. POLÍTICA DE ESCALATION configurável. CLASSIFICAÇÃO DE FALHA com sugestão anexada – operador não adivinha.



CAIXA-PRETA NÃO ESCALA EM PRODUÇÃO.

O PROBLEMA

Sem observabilidade séria, você fica refém de "o agente disse que tentou X".

- Não dá pra provar.
- Não dá pra refutar.
- Custo? Não dá pra atribuir.
- Decisão? Não dá pra rastrear.

DUAS CAMADAS NO SPEKTOR

1. Eventos tipados em event store auditável. Verificação iniciou. Judge respondeu. Gate falhou. Exportável pra Langfuse/OTel.

2. Harness Bundles – cada execução é **fingerprintada**: hash da config, dos prompts, do roteamento.

Você não diz mais "ficou mais lento desde terça". Diz: bundle a7f3... rodou em 12s, b9c1... em 38s.

Você não tem produto, tem fé.

COMO O SPEKTOR FAZ, NA PRÁTICA

Quem opera tem painel. Quem investiga incidente tem linha do tempo. Quem evolui o produto tem **VERSIONAMENTO CRIPTOGRÁFICO** do harness inteiro. Bundles alimentam SHADOW RUNS e PROMOTION (fracasso de alto sinal vira dataset versionado).



ERRO VIRA REGRA. REGRA VIRA PLAYBOOK.

O PROBLEMA

Log diz **o que falhou**. Playbook diz **o que repetir ou evitar antes do próximo tiro**.

A maioria dos agentes para no log. Você acumula 10.000 linhas de trace por semana e segue cometendo o mesmo erro porque a lição **nunca foi capturada como conhecimento reutilizável**.

O CONCEITO

Cognition Lesson = regra compacta gerada quando algo dá errado. Não é o trace inteiro. É:

- **Gatilho** (qual sinal disparou)
- **Anti-padrão** (o que ele fez de errado)
- **Padrão preferido** (o que fazer no lugar)
- **Prioridade e reincidência**

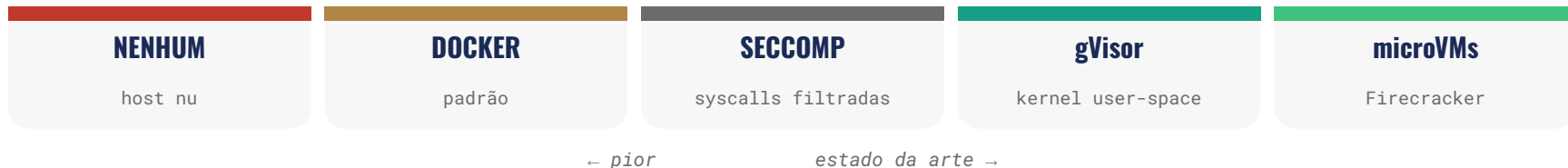
Duas trilhas: **Planning** (antes do executor) e **Execution** (quando gate falha).

COMO O SPEKTOR FAZ, NA PRÁTICA

Quando uma gate falha, a execução MONTA UM ADENDO CURTO com lições filtradas pelo gate. Erro que reincide tem contador. Lições recorrentes podem ser PROMOVIDAS pra memória de time – vira memória institucional. É o harness aprendendo em volta do modelo.



ONDE O CÓDIGO GERADO PELO AGENTE RODA?



POR QUE IMPORTA

Em 2026, OWASP elevou isso a risco de segurança: código gerado por LLM precisa rodar em sandbox real, não em Docker compartilhado.

Um agente pode gerar – intencional ou acidentalmente – código que apaga arquivos do host, exfiltra dados ou escala privilégio.

POSIÇÃO HONESTA DO SPEKTOR

Hoje o Spektor isola por subprocesso e por fronteiras de plugin. É melhor que rodar no host nu, mas não é isolamento de nível microVM.

O caminho natural – e está no radar – é mover pra uma microVM por execução, em execução autônoma de longa duração.



NÃO ESTAMOS TODOS NO MESMO JOGO.

FERRAMENTA	FOCO PRIMÁRIO	MODELO	DISPONIBILIDADE
Spektor	SDD pra software dev	Agent-agnostic, local-first	Em construção
Devin	Engenheiro autônomo end-to-end	Sandbox cloud proprietário	Closed · US\$ 500/mês
OpenClaw	Assistente pessoal multi-canal	Local + multi-LLM	Open source MIT
Hermes	Agente pessoal auto-melhorável	Local + multi-LLM	Open source MIT

OpenClaw e **Hermes** são assistentes pessoais – JARVIS multi-canal. Codam como efeito colateral. **Hermes** usa "Harness Engineering" no marketing, mas pra agente generalista. **Devin** é o concorrente direto no terreno de dev – fechado, cloud, sem SDD como espinha central. **Spektor** é o único cuja engine inteira é construída em volta de SDD pra dev de software.



A TABELA COMPLETA.

No cenário de desenvolvimento de software.

PILAR	SPEKTOR	DEVIN	OPENCLAW	HERMES
1 · Spec-Driven Execution	✓ Core, 4 fases	✗ Sem método	✗	✗
2 · Coordenação (Pilot)	✓ Pilot + Orchestrator	✓ Planner/Exec/Verifier	~ Multi-agent routing	~ Subagents
3 · Verification & Judge	✓ Calibrável	~ Interno	✗	~ Feedback layer
4 · Completion Gates	✓ Cascata 5 níveis	~ Testes	✗	✗
5 · Guardrails	✓ File + Change + Cost	✓ Sandbox cloud	~ Permissões	~ Constraint layer
6 · Resiliência	✓ Multi-gatilho + class.	✓ Recovery	~ Retry básico	~ Retry básico
7 · Observabilidade	✓ Event store + Bundles	~ Dashboard interno	~ Dashboard local	~ Trajectories

SÓ DO SPEKTOR Context Engine 98% · Memória 3 camadas · MCP/CLI/Daemon · agent-agnostic (Cursor, Claude Code, Copilot, Gemini CLI) · Wave/Swarm paralelo · Harness configurável · Local-first

AINDA EVOLUI Sandbox: hoje subprocesso + plugin; caminho é microVM.



OBRIGADO

O MODELO É COMMODITY. O HARNESS É A VANTAGEM COMPETITIVA.

Spektor vem aí. Fique de olho no Tech Leads Club pras próximas novidades.

Felipe Rodrigues

linkedin.com/in/felipfr

MAIO 2026